

Отчёт по лабораторной работе 5

АВТОР
Соловьев Богдан Михайлович, student

ПРИНАДЛЕЖНОСТЬ
Российский университет дружбы народов

```
using Pkg
Pkg.activate("C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\labs\lab05")
using Agents, DataFrames, Plots, StatsBase
println("Проект активирован: ", Base.active_project())
```

```
Проект активирован: C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\lab05
Activating project at `C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\lab05`
```

1 Цель работы

- Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
- Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.
- Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock.
- Модифицировать сеть, чтобы предотвратить deadlock.
- Проанализировать результаты и оформить отчёт с графиками и анимацией.

2 Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - juryter notebook;
 - документацию в формате Quarto.
- Выполнить код из juryter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - juryter notebook;
 - документацию в формате Quarto.
- Выполнить код из juryter notebook с параметрами.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

Сеть Петри есть математический аппарат для моделирования дискретных систем. Графически она представляется как двудольный ориентированный граф двух типов вершин: позиции (круги) и переходы (прямоугольники).

Позиции (Places) суть пассивные элементы, описывающие состояние системы (например, наличие ресурса или выполнение условия).

Внутри позиции могут находиться фишки (tokens) — неотрицательное целое число, указывающее на количество ресурсов.

Переходы (Transitions) суть активные элементы, описывающие события или действия системы. Они могут срабатывать, изменяя состояние модели.

Дуги (Arcs) суть направленные соединения между позициями и переходами (но не между двумя позициями или двумя переходами),

которые показывают, как состояние влияет на события и наоборот.

Маркировка (Marking) есть распределение фишек по позициям в определённый момент времени, то есть текущее состояние модели.

Смена маркировок происходит при срабатывании переходов в соответствии с определёнными правилами.

Теория сетей Петри, появившаяся как инструмент для анализа химических процессов, сегодня является мощным и наглядным математическим аппаратом.

Она незаменима везде, где нужно описать параллельные, асинхронные и распределённые системы. В её основе лежат всего четыре элемента, а богатство поведения возникает из их комбинация.

4 Выполнение лабораторной работы

Создаю пространство для выполнения лабораторной. Для этого создаю setup_report, потом add_packages и tangle. (Ничего нового),

потому перейдём сразу к модели. Создаю в папке src саму модель, которой буду пользоваться дальше. Теперь запускаю код, который

Создаёт классическую сеть Петри для N=5 философов с помощью build_classical_network.

и запускает симуляцию (алгоритм Гиллеспи) на время tmax = 50.0.

Анализирует последнее состояние на предмет deadlock с помощью detect_deadlock и выводит результат в консоль.

есть ли дедлок(рис. 1).

```
ing/labs/lab05/project/scripts (master)
$ julia test.jl
Activating project at `C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\labs\lab05\project`
==== Классическая сеть (без арбитра) ====
deadlock обнаружен: true
==== Сеть с арбитром ====
deadlock обнаружен: false
```

Рисунок 1: дедлок

Так же этот скрипт выводит графики обычной модели (рис. 2).

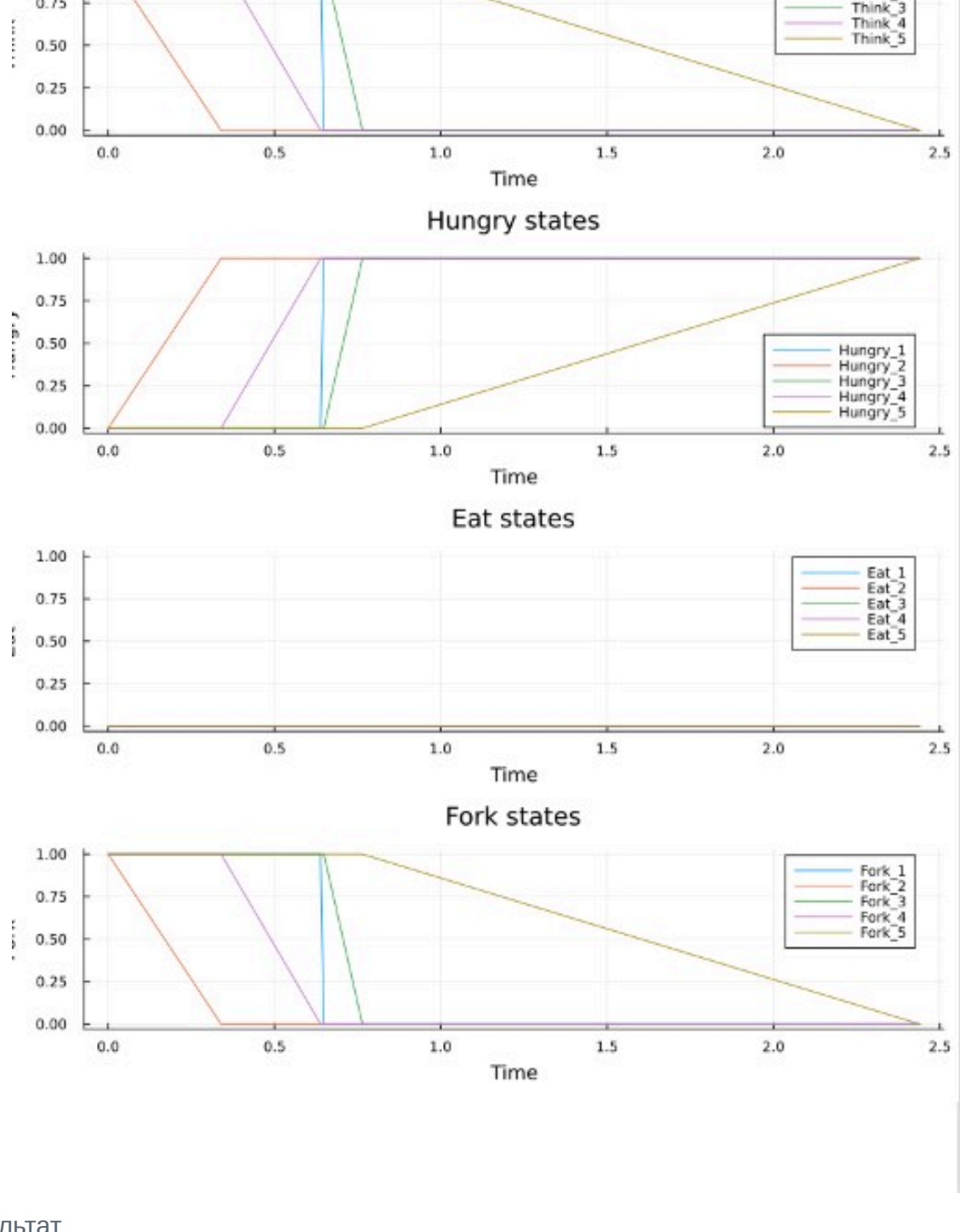


Рисунок 2: Результат

и модели с арбитром (рис. 3).

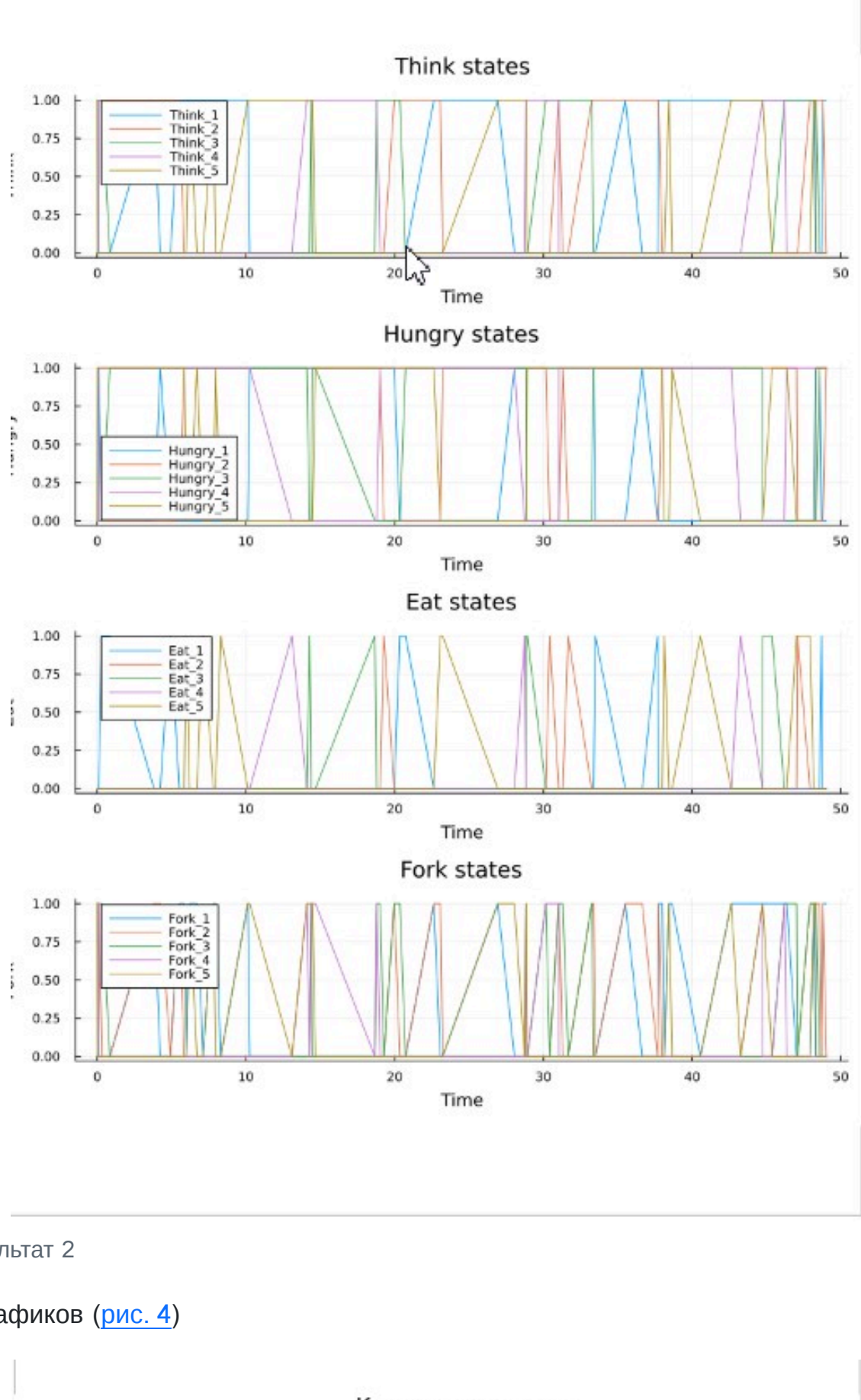


Рисунок 3: результат 2

Сравнение графиков (рис. 4)

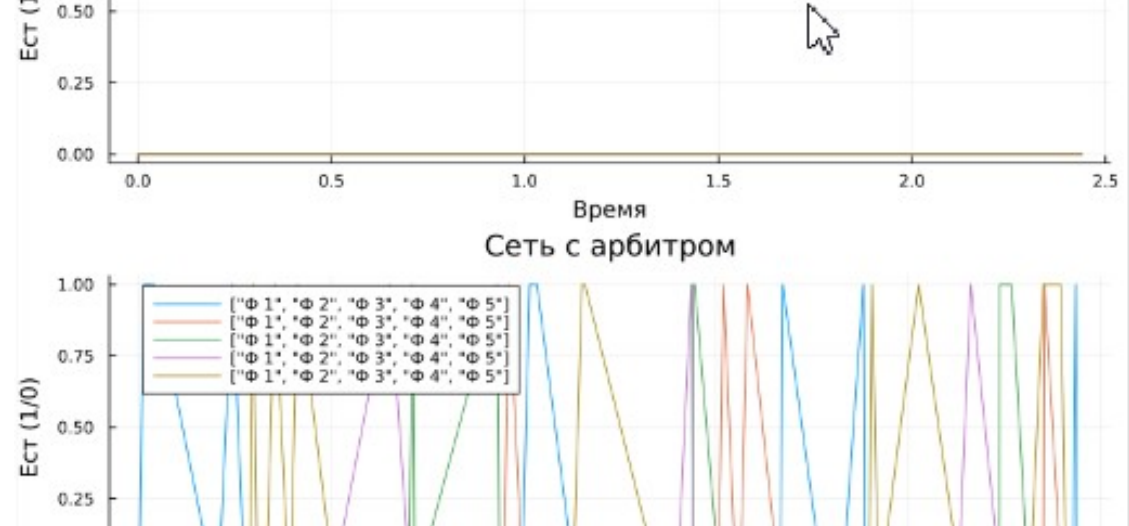


Рисунок 4: parameters

title: "Моделирование задачи об обедающих философах" format: html —

Демонстрация стохастического поведения двух сетей Петри: 1. Классическая сеть (подвержена deadlock) 2. Сеть с арбитром (свободна от deadlock)

4.1 Подготовка окружения

```
using DrWatson
@quickactivate "project"
include(srcdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots

julia> Warning: DrWatson could not find a project file by recursively checking given `dir`
(given dir: C:\Users\bogda)

julia> @ DrWatson C:\Users\bogda\julia\packages\DrWatson\2QF5p\src\project_setup.jl:84

WARNING: replacing module DiningPhilosophers.

WARNING: using DiningPhilosophers.build_classical_network in module Main conflicts with ar
WARNING: using DiningPhilosophers.build_arbiter_network in module Main conflicts with an e
WARNING: using DiningPhilosophers.simulate_stochastic in module Main conflicts with an exi
WARNING: using DiningPhilosophers.detect_deadlock in module Main conflicts with an existi
WARNING: using DiningPhilosophers.plot_marking_evolution in module Main conflicts with an
```

4.2 Параметры моделирования

```
N = 5 # Количество философов
tmax = 50.0 # Время моделирования
```

50.0

4.3 1. Классическая сеть (без арбитра)

В классической постановке каждый философ захватывает левую вилку, затем правую. При совпадении моментов захвата возможен deadlock.

```
println("==== Классическая сеть (без арбитра) ====")
net_classic, u0_classic, _ = build_classical_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)

==== Классическая сеть (без арбитра) ====

18x21 DataFrame
Row time Think_1 Think_2 Think_3 Think_4 Think_5 Hungry_1 Hungry_2 Hungry_3 Hungry_4 Hungry_5 E
Float64 Float64 Float64 Float64 Float64 Float64 Float64 Float64 Float64 Float64 Float64 F
1 0.0 1.0 1.0 1.0 1.0 1.0 0.0 0.0 0.0 0.0 0.0 0.0
2 0.0247648 1.0 1.0 1.0 1.0 1.0 0.0 0.0 0.0 0.0 0.0 1.0
3 0.532218 1.0 1.0 1.0 1.0 1.0 0.0 0.0 0.0 0.0 0.0 0.0
4 0.971786 1.0 1.0 0.0 1.0 1.0 0.0 0.0 0.0 1.0 0.0 0.0
5 1.217137 1.0 0.0 1.0 1.0 1.0 0.0 0.0 1.0 1.0 0.0 0.0
6 1.2729 1.0 0.0 0.0 0.0 1.0 0.0 0.0 1.0 1.0 1.0 0.0
7 1.69335 1.0 0.0 0.0 0.0 1.0 0.0 0.0 1.0 1.0 1.0 0.0
8 1.72209 1.0 0.0 0.0 0.0 1.0 0.0 0.0 1.0 1.0 0.0 0.0
9 2.63334 1.0 0.0 0.0 1.0 1.0 0.0 1.0 1.0 0.0 0.0 0.0
10 2.75181 1.0 0.0 0.0 1.0 1.0 0.0 1.0 1.0 0.0 1.0 0.0
11 3.45197 0.0 0.0 0.0 1.0 1.0 0.0 1.0 1.0 0.0 1.0 0.0
12 3.6564 0.0 0.0 0.0 1.0 1.0 0.0 1.0 1.0 0.0 0.0 1.0
13 4.98901 0.0 0.0 1.0 1.0 0.0 0.0 1.0 1.0 0.0 0.0 0.0
14 5.03416 0.0 0.0 1.0 0.0 0.0 1.0 1.0 0.0 1.0 0.0 1.0
15 5.25979 0.0 0.0 1.0 0.0 0.0 1.0 0.0 0.0 0.0 1.0 1.0
16 6.64102 0.0 1.0 1.0 0.0 0.0 1.0 0.0 0.0 1.0 1.0 0.0
17 6.769 0.0 0.0 1.0 0.0 0.0 1.0 1.0 0.0 1.0 1.0 0.0
18 7.33636 0.0 0.0 0.0 0.0 0.0 1.0 1.0 1.0 1.0 1.0 1.0
```

Сохранение результатов

```
CSV.write(datadir("dining_classic.csv"), df_classic)

"C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\labs\lab05\p
```

Проверка на deadlock

```
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock обнаружен: sdead")

Deadlock обнаружен: true
```

Визуализация эволюции маркировки

```
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotsdir("classic_simulation.png"))

"C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\labs\lab05\p
```

4.4 2. Сеть с арбитром

Арбитр управляет очередностью захвата вилок, исключая возникновение циклического ожидания.

```
println("\n==== Сеть с арбитром ====")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)

==== Сеть с арбитром ====
```

Row	time	Think_1	Think_2	Think_3	Think_4	Think_5	Hungry_1	Hungry_2	Hungry_3	Hungry_4	Hungry_5	E
		Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64
1	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0520744	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0
3	0.0565488	1.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0	1.0	0.0	0.0
4	0.0937051	0.0	1.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0	0.0
5	0.462059	0.0	1.0	0.0	0.0	1.0	1.0	0.0	1.0	0.0	0.0	0.0
6	0.48095	0.0	1.0	0.0	1.0	1.0	1.0	0.0	1.0	0.0	0.0	1.0
7	0.501209	0.0	1.0	0.0	0.0	1.0	1.0	0.0	1.0	0.0	1.0	0.0
8	1.17419	0.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0	1.0	0.0	1.0
9	1.57926	0.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0
10	4.10927	1.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	1.0	0.0
11	4.29765	0.0	1.0	0.0	0.0	1.0	1.0	0.0	1.0	0.0	1.0	0.0
12	4.34083	0.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	1.0	0.0
13	4.55964	1.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	1.0	0.0
14	5.03416	0.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0	1.0
15	5.25979	0.0	0.0	1.0	0.0	0.0	1.0	0.0	1.0	0.0	0.0	1.0
16	6.64102	0.0	1.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	1.0
17	6.769	0.0	0.0	1.0	0.0	0.0	1.0	1.0	0.0	1.0	0.0	1.0
18	7.33636	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0

Сохранение результатов

```
CSV.write(datadir("dining_arbiter.csv"), df_arb)

"C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\labs\lab05\p
```

Проверка на deadlock (ожидается false)

```
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock обнаружен: sdead_arb")

Deadlock обнаружен: false
```

Визуализация эволюции маркировки

```
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotsdir("arbiter_simulation.png"))

"C:\Users\bogda\work\study\2026-1\2026-1--study--simulationmodeling\labs\lab05\p
```

4.5 Выводы

- Классическая сеть демонстрирует возможность deadlock при стохастическом моделировании.
- Сеть с арбитром гарантированно избегает deadlock.
- Результаты сохранены в data/ и plots/.

Для повторного анализа загрузите данные командой:

```
df = CSV.read(datadir("dining_classic.csv"), DataFrame)
```

5 Выводы

Я построил сеть Петри для пяти философов и модифицировал сеть с помощью арбитра, чтобы предотвратить deadlock

Список литературы

Повторное использование

CC BY 4.0

Содержание

- 1 Цель работы
- 2 Задание
- 3 Теоретическое введение
- 4 Выполнение лабораторной работы
- 5 Выводы
- Список литературы